

INTRODUCTION TO DESIGN AND ANALYSIS OF ALGORITHMS

Algorithm

Informally, an **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**. An algorithm is thus a sequence of computational steps that transform the input into the output.

We can also view an algorithm as a tool for solving a well-specified **computational problem**. The statement of the problem specifies in general terms the desired input/output relationship. The algorithm describes a specific computational procedure for achieving that input/output relationship.

An algorithm is said to be **correct** if, for every input instance, it halts with the correct output. We say that a correct algorithm **solves** the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with other than the desired answer. Contrary to what one might expect, incorrect algorithms can sometimes be useful, if their error rate can be controlled.

Characteristics/properties/features of an Algorithm

An algorithm must have the following properties:

- **Unambiguous** /definiteness– Algorithm should be clear and unambiguous. Each of its steps (or phases), and their input/outputs should be clear and must lead to only one meaning.
- **Input** – An algorithm should have 0 or more well defined inputs.
- **Output** – An algorithm should have 1 or more well defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Feasibility** – Should be feasible with the available resources.
- **Independent** – An algorithm should have step-by-step directions which should be independent of any programming code.
- **Generality**: The algorithm should work for all problems of the desired form.

Types of algorithms:

1. Simple recursive algorithms. Ex: Searching an element in a list
2. Backtracking algorithms Ex: Depth-first recursive search in a tree
3. Divide and conquer algorithms. Ex: Quick sort and merge sort
4. Dynamic programming algorithms. Ex: Generation of Fibonacci series
5. Greedy algorithms Ex: Counting currency

6. Branch and bound algorithms. Ex: Travelling salesman (visiting each city once and minimize the total distance travelled)
7. Brute force algorithms. Ex: Finding the best path for a travelling salesman
8. Randomized algorithms. Ex. Using a random number to choose a pivot in quick sort).

Analysis of an algorithm

An algorithm must not only be able to solve the problem at hand, it must be able to do so in as efficient a manner as possible. There might be different ways in which we can solve a given problem. The characteristics of each algorithm will determine how efficiently each will operate. Determining which algorithm is efficient than the other devices involves analysis of algorithms.

Analysis of an algorithm provides information that gives us a general idea of how long an algorithm will take for solving a problem.

Analysis of algorithms is therefore the process of determining, as precisely as possible, how many resources (such as time and memory) an algorithm consumes when it executes.

To analyze the amount of work done by an algorithm, we will produce measures that express a count of the operations done by an algorithm as a function of the size of the input to the algorithm.

Stages of analysis

Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation, as mentioned below –

1. ***A priori analysis*** – This is theoretical analysis of an algorithm. Efficiency of algorithm is measured by assuming that all other factors e.g. processor speed, are constant and have no effect on implementation.
2. ***A posteriori analysis*** – This is empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required, are collected. The Goals of *empirical analysis* include:

- Obtain insights into algorithmic performance.
- Help assessing suitability for applications.
- Provide basis for comparing algorithms.
- Characterize algorithm behavior.
- Facilitate improvements of algorithms.

MEASURING AND ANALYSING ALGORITHM COMPLEXITY

Algorithmic complexity is concerned about how fast or slow particular algorithm performs.

Complexity is the number of steps required to solve a problem provided each such step takes constant time. The goal is to find the best algorithm to solve the problem with a less number of steps. We define complexity as a numerical function $T(n)$ - time versus the input size n . We want to define time taken by an algorithm without depending on the implementation details.

(i) Time Complexity:

Time complexity is a function describing the amount of time an algorithm takes in terms of the amount of input to the algorithm. "Time" can mean the number of:

- memory accesses performed,
- the number of comparisons between integers,
- the number of times some inner loop is executed,
- or some other natural unit related to the amount of real time the algorithm will take.

We try to keep this idea of time separate from "wall clock" time, since many factors unrelated to the algorithm itself can affect the real time (like the language used, type of computing hardware, proficiency of the programmer, optimization in the compiler, etc.). It turns out that, if we chose the units wisely, all of the other stuff doesn't matter and we can get an independent measure of the efficiency of the algorithm.

Time requirements can be defined as a numerical function $\mathbf{T(n)}$, where $\mathbf{T(n)}$ can be measured as the number of steps, provided each step consumes constant time.

For example, addition of two n -bit integers takes n steps. Consequently, the total computational time is $\mathbf{T(n) = c*n}$, where c is the time taken for addition of two bits. Here, we observe that $\mathbf{T(n)}$ grows linearly as input size increases.

(ii) Space Complexity:

Space Complexity is a function describing the amount of memory (space) an algorithm takes in terms of the amount of input to the algorithm. .

Space required by an algorithm is equal to the sum of the following two components:

- A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example simple variables & constant used, program size etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example dynamic memory allocation, recursion stack space etc.

Space complexity $S(P)$ of any algorithm P is $S(P) = C + SP(I)$ Where C is the fixed part and $S(I)$ is the variable part of the algorithm which depends on instance characteristic I .

Aims of analyzing algorithm complexity

The aim of measuring and analyzing algorithm complexity is three fold:

- i. To look at various ways of comparing algorithms;
- ii. To look at the idea of expressing the running time of an algorithm as a function of input size;
- iii. To look at the idea of defining the worst-case, best-case and average-case running times.

CASES TO CONSIDER DURING ANALYSIS

Choosing the input to consider when analyzing an algorithm can have a significant impact on how an algorithm will perform. For example, if the input list is already sorted, some sorting algorithms will perform very well, but other sorting algorithms may perform poorly. The opposite may be true if the list is randomly arranged instead of sorted. There different cases to be considered during analysis this include:

- Best Case
- Worst Case
- Average case

BEST CASE

This represents the input set that allows an algorithm to perform most quickly. With this input the algorithm takes the shortest time to execute, as it causes the algorithm to do the least amount of work. For example, for a searching algorithm the best case would be if the value we are searching for is found in the first location that the search algorithm checks. As a result, this algorithm would need only one comparison irrespective of the complexity of the algorithm. No matter how large is the input, searching in best case will result in a constant time of 1. Since the best case for an algorithm would usually be very small and frequently constant value, a best case analysis is often not done.

WORST CASE

Worst case is an important analysis because it gives us an idea of the most time an algorithm will ever take. Worst case analysis requires that we identify the input values that cause an algorithm to do the most work. For example, for a searching algorithm, the worst case is one where the value is in the last place we check or is not in the list. This could involve comparing the key to each list value for a total of N comparisons.

AVERAGE CASE

This represents the input set that allows an algorithm to deliver average performance. Doing Average-case analysis is a four-step process:

- Determine the number of different groups into which all possible input sets can be divided
- Determine the probability that the input will come from each of these groups
- Determine how long the algorithm will run for each of these groups. All of the input in each group should take the same amount of time, and if they do not, the group must be split into two separate groups.

ASYMPTOTIC ANALYSIS

In Asymptotic Analysis, we evaluate the performance of an algorithm in terms of input size (we don't measure the actual running time). Asymptotic analysis attempts to estimate the resource consumption of an algorithm. It allows us to compare the relative costs of two or more algorithms for solving the same problem. Asymptotic analysis also gives algorithm designers a tool for estimating whether a proposed solution is likely to meet the resource constraints for a problem before they implement an actual program.

Asymptotic analysis measures the efficiency of an algorithm, or its implementation as a program, as the input size becomes large. It is actually an estimating technique and does not tell us anything about the relative merits of two programs where one is always "slightly faster" than the other. However, asymptotic analysis has proved useful to computer scientists who must determine if a particular algorithm is worth considering for implementation.

The Purpose of Asymptotic Analysis

- To estimate how long a program will run.
- To estimate the largest input that can reasonably be given to the program.
- To compare the efficiency of different algorithms.
- To help focus on the parts of code that are executed the largest number of times.
- To choose an algorithm for an application.

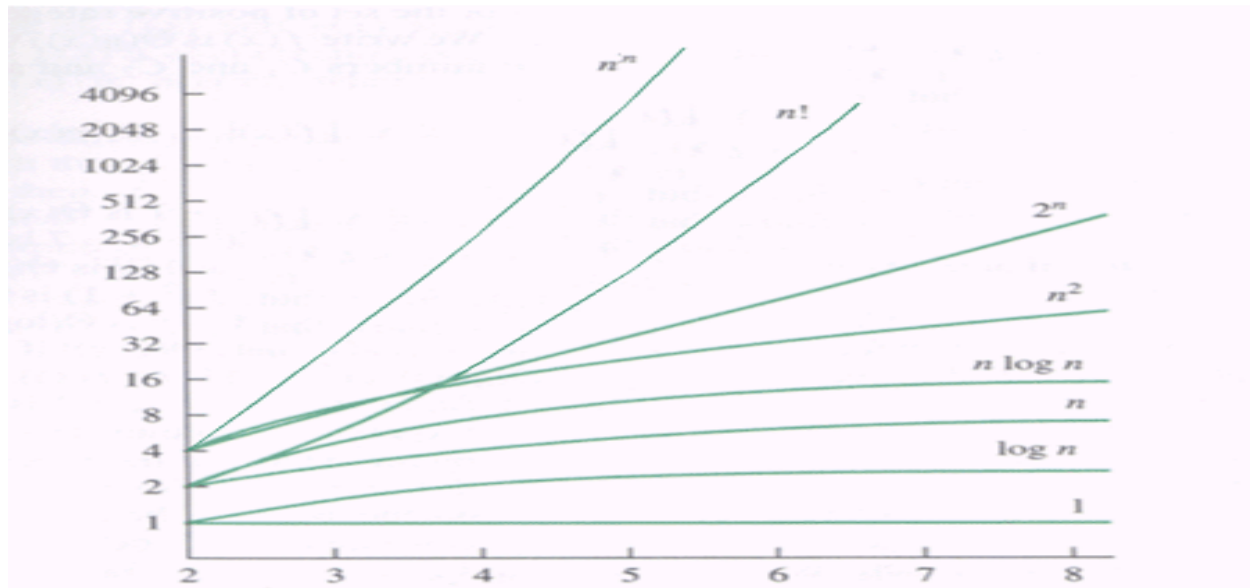
Measurements of Running time

Running time of an algorithm is almost always *independent* of the programming language, or even the methodology we use. It is convenient to use a function $T(n)$ to represent the number of units of time taken by a program or an algorithm on any input of size n . We shall call $T(n)$ the running time of the program. For example, a program may have a running time $T(n) = cn$, where c is some constant. Put another way, the running time of this program is linearly proportional to the size of the input on which it is run. Such a Linear-time program or algorithm is said to be linear time, or just linear.

The different running times of programs or algorithms are summarized in the table below:

Log n	When the running time of a program is logarithmic, the program gets slightly slower as n grows. This running time commonly occurs in programs that solve a big problem by transforming it into a smaller problem, cutting the size by some constant fraction., When n is a million, log n is a doubled. Whenever n doubles, log n increases by a constant, but log n does not double until n increases to n^2 .
n	When the running time of a program is linear, it is generally the case that a small amount of processing is done on each input element. This is the optimal situation for an algorithm that must process n inputs.
n log n	This running time arises for algorithms that solve a problem by breaking it up into smaller sub-problems, solving then independently, and then combining the solutions. When n doubles, the running time more than doubles.
n²	When the running time of an algorithm is quadratic, it is practical for use only on relatively small problems. Quadratic running times typically arise in algorithms that process all pairs of data items (perhaps in a double nested loop) whenever n doubles, the running time increases four fold.
n³	Similarly, an algorithm that process triples of data items (perhaps in a triple-nested loop) has a cubic running time and is practical for use only on small problems. Whenever n doubles, the running time increases eight fold.
2ⁿ	Few algorithms with exponential running time are likely to be appropriate for practical use, such algorithms arise naturally as “brute-force” solutions to problems. Whenever n doubles, the running time squares.

This are depicted in the diagram below:



Rates of Growth

The theory of the asymptotic growth rate of functions, also called the order of growth of functions, provides a basis for partitioning algorithms into groups with equivalent efficiency. In comparing the efficiency of algorithms, we are more interested in big differences that manifest themselves as the size of the input becomes large than we are in small differences in running times that vary by a constant or a multiple for inputs of all sizes. Based on their order algorithm are grouped into three categories:

- Big Oh O
- Big Omega Ω
- Big theta Θ

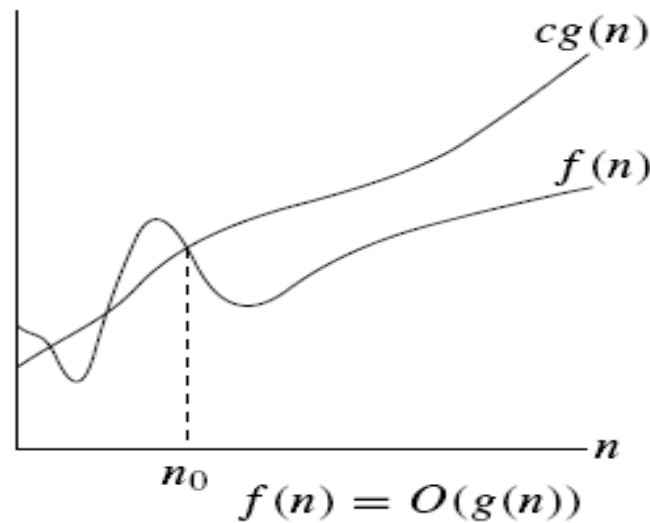
Big Oh (Big-O)

This represents a class of functions that grow at the same rate. big O notation describes upper bounds on a function $f(n)$. It measures the worst case time complexity or longest amount of time an algorithm can possibly take to complete.

Knowing where a function falls within the big-O hierarchy allows us to compare it quickly with other functions and gives us an idea of which algorithm has the best time performance.

Definition: Let $f(n)$ and $g(n)$ be functions, where n is a positive integer. We write $f(n) = O(g(n))$ if and only if there exists a real number c and positive integer n_0 satisfying $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$. (And we say, "f of n is big oh of g of n." We might also say or write $f(n)$ is $inO(g(n))$).

because we can think of O as a *set* of functions all with the same property. But we won't often do that in Data Structures.)



While considering two algorithms we will want to know if the function categorizing the behavior of the first is in big oh of the second. If so, we know that the second algorithm does no better than the first in solving the problem.

Examples:

1. Show $3n^2 + 4n - 2 = O(n^2)$.

We need to find c and n_0 such that:

$$3n^2 + 4n - 2 \leq cn^2 \text{ for all } n \geq n_0.$$

Divide both sides by n^2 , getting:

$$3 + 4/n - 2/n^2 \leq c \text{ for all } n \geq n_0.$$

If we choose n_0 equal to 1, then we need a value of c such that:

$$3 + 4 - 2 \leq c$$

We can set c equal to 6. Now we have:

$$3n^2 + 4n - 2 \leq 6n^2 \text{ for all } n \geq 1.$$

2. $n^2 + 42n + 7 = O(n^2)$

We need to find c and n_0 such that:

$$n^2 + 42n + 7 \leq cn^2 \text{ for all } n \geq n_0.$$

Divide both sides by n^2 , getting:

$$1 + 42/n - 7/n^2 \leq c \text{ for all } n \geq n_0.$$

If we choose n_0 equal to 1, then we need a value of c such that:

$$1 + 42 + 7 \leq c$$

We can set c equal to 50. Now we have:

$$3n^2 + 4n - 2 \leq 6n^2 \text{ for all } n \geq 1.$$

$$n^2 + 42n + 7 \leq 50n^2 \text{ for all } n \geq 1.$$

3. Prove that running time $T(n) = n^3 + 20n + 1$ is $O(n^3)$

We need to find c and n_0 such that:

$$n^3 + 20n + 1 \leq cn^3 \text{ for all } n \geq n_0.$$

Divide both sides by n^3 , getting:

$$1 + 20/n^2 - 1/n^3 \leq c \text{ for all } n \geq n_0.$$

If we choose n_0 equal to 1, then we need a value of c such that:

$$1 + 20 + 1 \leq c$$

We can set c equal to 22. Now we have:

$$n^3 + 20n + 1 \leq 22n^3 \text{ for all } n \geq 1.$$

The following functions are often encountered in computer science Big Oh analysis:

- $T(n) = O(1)$. This is called constant growth. $T(n)$ does not grow at all as a function of n , it is a constant. It is pronounced "Big Oh of one." For example, array access has this characteristic. $A[i]$ takes the same time independent of the size of the array A .
- $T(n) = O(\lg(n))$. This is called logarithmic growth. $T(n)$ grows proportional to the base 2 logarithm of n . Actually, the base doesn't matter, it's just traditional to use base-2 in

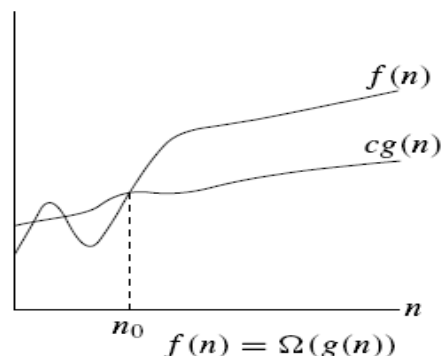
computer science. It is pronounced "Big Oh of log n." For example, binary search has this characteristic.

- $T(n) = O(n)$. This is called linear growth. $T(n)$ grows linearly with n . It is pronounced "Big Oh of n ." For example, looping over all the elements in a one-dimensional array would be an $O(n)$ operation.
- $T(n) = O(n \log n)$. This is called " $n \log n$ " growth. $T(n)$ grows proportional to n times the base 2 logarithm of n . It is pronounced "Big Oh of $n \log n$." For example, Merge Sort has this characteristic. In fact, no sorting algorithm that uses comparison between elements can be faster than $n \log n$.
- $T(n) = O(n^k)$. This is called polynomial growth. $T(n)$ grows proportional to the k -th power of n . We rarely consider algorithms that run in time $O(n^k)$ where k is greater than 5, because such algorithms are very slow. For example, selection sort is an $O(n^2)$ algorithm. It is pronounced "Big Oh of n squared."
- $T(n) = O(2^n)$. This is called exponential growth. $T(n)$ grows exponentially. It is pronounced "Big Oh of 2 to the n ." Exponential growth is the most-feared growth pattern in computer science; algorithms that grow this way are basically useless for anything but very small problems.

Big-Omega notation (Big-Ω)

Represents a class of functions that grow at least as fast as some function. Big-Omega notation bounds a function from below instead of from above. It measures the best case time complexity or best amount of time an algorithm can possibly take to complete.

A function $f(n)$ is $\Omega(g(n))$ if there are values c and n_0 such that $f(n) \geq c g(n)$ for all $n > n_0$.



Notice the $\leq$ in big-O notation has been replaced with a $\geq$ in big-Omega notation. Although big-O notation is mostly used to analyze "algorithms", big-Omega notation is mostly used to analyze "problems". With big-O notation we analyze one specific algorithm/method to determine an upper bound on its performance. In big-Omega notation we analyze all possible algorithms/methods to determine an lower bound on performance. This second task is much harder.

Small-o

Small-o, commonly written as **o**, is an Asymptotic Notation to denote the upper bound (that is not asymptotically tight) on the growth rate of runtime of an algorithm.

$f(n)$ is $o(g(n))$, if for any real constant c ($c > 0$), $f(n)$ is $< c g(n)$ for every input size n ($n > 0$).

The definitions of O -notation and o -notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $f(n) \leq g(n)$ holds for **some** constant $c > 0$, but in $f(n) = o(g(n))$, the bound $f(n) < c g(n)$ holds for **all** constants $c > 0$.

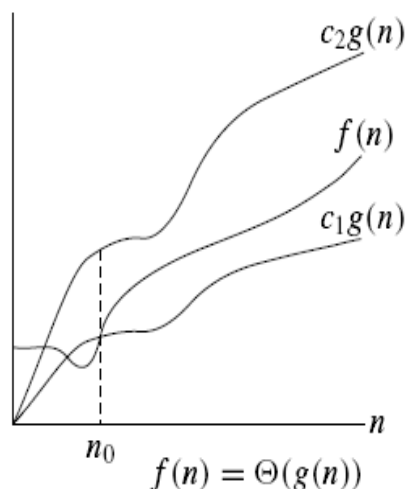
Small-omega

Small-omega, commonly written as **ω** , is an Asymptotic Notation to denote the lower bound (that is not asymptotically tight) on the growth rate of runtime of an algorithm.

$f(n)$ is $\omega(g(n))$, if for any real constant c ($c > 0$), $f(n)$ is $> c g(n)$ for every input size n ($n > 0$).

The definitions of Ω -notation and ω -notation are similar. The main difference is that in $f(n) = \Omega(g(n))$, the bound $f(n) \geq g(n)$ holds for **some** constant $c > 0$, but in $f(n) = \omega(g(n))$, the bound $f(n) > c g(n)$ holds for **all** constants $c > 0$.

Big Theta notation (Big- Θ)



Big Theta notation is used to describe pairs of functions that have equivalent asymptotic growth

rates. Big-Theta notation is a combination of big-O and big-Omega, which bounds a function from below and above. is the formal way to express both the lower bound and upper bound of an algorithm's running time.

A function $f(n)$ is $\Theta(g(n))$ if there are values c_1 , c_2 , and n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n > n_0.$$